CAMPUS EVENT MANAGEMENT SYSTEM

Software Requirements Specification & Project Understanding

Prepared from the supplied capstone project source code

Application type	Web-based event discovery, booking, approval and check-in system
Frontend	React 18, Vite, React Router
Backend	Python, Flask REST API, JWT authentication
Database	MongoDB
Deployment support	Dockerfiles and Docker Compose configuration

Document purpose: explain what the current implementation is intended to do, its main requirements, roles, workflows, architecture, constraints and verification points.

Document guide

• 1. Executive summary

• 2. Project understanding and objectives

• 3. Scope and user roles

• 4. Functional requirements

• 5. Main workflows and business rules

• 6. System architecture and technology

• 7. Data model overview

• 8. API overview

• 9. Non-functional requirements

• 10. Assumptions, constraints and risks

• 11. Acceptance criteria and test scenarios

• 12. Suggested future improvements

1. Executive summary

The Campus Event Management System is a role-based web application for managing campus events from creation through attendance. Students can discover approved events, filter/search the event feed, reserve a seat, join a waitlist when capacity is reached, view their tickets and cancel eligible bookings. Organizers can create and manage event drafts, submit events for review, monitor event statistics and validate tickets at check-in. Administrators review submissions, manage venues and manage user accounts and roles.

The project separates a React frontend from a Flask API and uses MongoDB for persistence. JWT access and refresh tokens support authenticated sessions, while role checks restrict actions to the relevant user type. Ticket booking and cancellation logic uses MongoDB sessions/transactions to coordinate capacity and ticket changes.

> Important scope note: this document describes functionality visible in the supplied source. It is not a claim that every production-grade control has been implemented or independently tested.

2. Project understanding and objectives

Problem being addressed

Campus events can involve multiple disconnected tasks: publishing event details, controlling capacity, reviewing event proposals, issuing tickets, handling full events and checking attendees in. The system centralizes these tasks and gives each role a focused workflow.

Primary objectives

• Provide one place for students to browse approved upcoming events and inspect event/venue details.

• Prevent ordinary users from performing organizer or administrator actions.

• Enforce event capacity and prevent duplicate active bookings for the same student and event.

• Support a queue when an event is full and promote the next eligible attendee after a cancellation.

• Give organizers event-level booking, cancellation, waitlist and check-in metrics.

• Give administrators an approval workflow and tools for venue and account administration.

• Provide a maintainable frontend/API separation and container-based development setup.

3. Scope and user roles

In scope

• Account registration and login; password hashing; access/refresh JWT flow; logout/revocation.

• Role-specific screens and protected routes for students, organizers and administrators.

• Event search, category and date filtering, event details and venue information.

• Event draft creation, editing/deletion under permitted conditions, and submission for approval.

• Admin approval/rejection with review notes.

• Seat booking, ticket cancellation, waitlisting, queue position maintenance and ticket history.

• Ticket-code/QR presentation and organizer check-in validation.

• Venue management, user listing/creation and role changes.

• Organizer dashboard statistics and health endpoint.

Out of scope or not evidenced in the supplied implementation

• Payment processing, refunds or paid ticketing.

• Email/SMS notifications and calendar integrations.

• External identity providers or multi-factor authentication.

• Native mobile applications.

• Automated end-to-end test suite, monitoring/alerting, backup/restore procedures or formal service-level objectives.

User roles

Role	Capabilities	Restrictions / boundaries
Student	Register/login; browse and filter approved events; view details; book or waitlist; view ticket history; cancel eligible ticket or leave waitlist.	Cannot create/approve events, manage venues/users or check in attendees.
Organizer	Create and edit eligible drafts; submit for review; view own events and dashboard; check in tickets for own events.	Cannot approve own event or administer all users/venues. Event ownership is checked for organizer operations.
Administrator	Review pending events; approve/reject; manage venues; list/create users and change roles.	Administrative endpoints require an authenticated admin role.

4. Functional requirements

FR-01 Account registration

The system shall allow a new user to register with a name, email and password of at least eight characters. New public registrations receive the student role. Email addresses are normalized to lowercase; duplicate email records should be rejected.

FR-02 Authentication

The system shall verify login credentials and return access and refresh tokens plus non-password user details. Invalid credentials shall return an authentication error.

FR-03 Session/token handling

The system shall support refresh-token renewal and logout/revocation. Authenticated endpoints shall reject missing, invalid or expired tokens.

FR-04 Role authorization

The system shall restrict student, organizer and admin API operations by role. The frontend shall protect role-specific routes and show an unauthorized/not-found experience when appropriate.

FR-05 Event discovery

Students shall be able to list approved events that have not ended, search event title/description, filter by category and apply date filters.

FR-06 Event details

Students shall be able to view event details, venue information, remaining seats and waitlist information.

FR-07 Event draft management

Organizers shall be able to create an event draft with title, description, category/tags, venue, capacity and start/end time. The event capacity must be positive and not exceed venue capacity; end time must follow start time.

FR-08 Event submission

Organizers shall be able to submit eligible drafts for admin review. Submission changes the event status to pending.

FR-09 Event review

Administrators shall be able to inspect pending events and approve or reject them with an optional review note.

FR-10 Booking

A student shall be able to book an available seat on an approved, not-ended event. The system shall not permit duplicate active/used tickets for the same student and event.

FR-11 Waitlist

If an event has no available seats, the system shall allow the student to join a waitlist and provide a queue position. Duplicate waitlist entries shall be prevented.

FR-12 Cancellation and promotion

A student shall be able to cancel an eligible booked ticket. Capacity shall be adjusted, and the earliest waitlisted student shall be promoted when applicable.

FR-13 Ticket history

Students shall be able to view their tickets and associated event details, including status.

FR-14 Check-in

Organizers shall be able to submit a ticket code to mark a booked ticket as used. The ticket must belong to an event owned by that organizer and must not already be used/cancelled.

FR-15 Organizer dashboard

The system shall report capacity, occupied seats, cancellations, check-ins, check-in rate and waitlist length for an organizer's event.

FR-16 Venue administration

Administrators shall be able to create, list, update and delete venues, subject to input validation.

FR-17 User administration

Administrators shall be able to list users without exposing password hashes, create users with valid roles and change user roles.

FR-18 Health check

The backend shall expose a health endpoint that checks database connectivity and returns an unavailable response if the database ping fails.

5. Main workflows and business rules

Event lifecycle

Expected flow: organizer creates a draft → organizer submits it → status becomes pending → administrator approves or rejects it → approved events become visible to students. Rejected events can be edited and resubmitted. Only eligible draft/rejected events can be edited or deleted; the implementation also prevents deletion where tickets have been sold.

Booking lifecycle

A student opens an approved event and requests a booking. The backend checks event status/time and whether the student already has a ticket. If capacity is available, it increments occupied seats and creates a ticket. If full, it creates a waitlist entry. When a booked ticket is cancelled, the seat is released and the first waiting student can be promoted into a ticket.

Ticket lifecycle

The principal ticket states observed in the code are booked, used and cancelled. A booked ticket may be cancelled by its owner or marked used by an organizer at check-in. A used/cancelled ticket must not be checked in again. A student cannot obtain another ticket for an event if an existing ticket is booked or used.

Key business rules

• Only approved events should appear in the student event feed.

• Event start/end times must be valid; end time must be later than start time.

• Event capacity must be positive and cannot exceed the selected venue capacity.

• Booking and waitlisting close when the event has ended.

• Organizer operations must be limited to events owned by that organizer.

• Passwords are stored as hashes rather than plaintext.

• Ticket codes must be unique; user emails and venue names have unique indexes.

• Database transaction support is required for the transaction-based booking/cancellation paths.

6. System architecture and technology

Layer	Technology / responsibility
Presentation	React 18 and Vite; React Router pages; shared layout/components; auth context; fetch-based API helper; QR/ticket UI and QR scanning library.
API / business logic	Python Flask application with Blueprints for authentication, students, organizers and administrators. JWT-based authentication and role authorization.
Persistence	MongoDB collections for users, venues, events, tickets, waitlist, check-ins and token blocklist. Indexes enforce selected uniqueness rules.
Runtime / packaging	Separate frontend/backend Dockerfiles and a Docker Compose configuration exposing frontend and backend ports.

High-level request flow

Browser (React UI) → API helper (fetch; JWT attached where needed) → Flask route/Blueprint → authentication and role checks → MongoDB query/update → JSON response → React updates the page.

Configuration note: the frontend Vite configuration proxies /api requests to the backend during development. Environment configuration is used for API/database/token settings. Secrets must be supplied through environment variables and must not be committed to source control.

7. Data model overview

Collection	Likely key fields observed	Purpose
users	_id, name, email, password_hash, role, created_at	Accounts, authentication and role-based access.
venues	_id, name, location, capacity	Physical/event venues and maximum venue capacity.
events	_id, title, description, category, tags, start_time, end_time, venue_id, capacity, seats_sold, status, organizer_id	Event details, lifecycle status, owner and capacity counters.
tickets	_id, event_id, student_id, code, status, booked_at, cancelled_at, checked_in_at	Student booking and check-in record.
waitlist	_id, event_id, student_id, position, created_at	Queue entries for full events.
checkins	_id, ticket_id, scanned_by, timestamp	Audit-style record of attendee check-in.
token_blocklist	_id, jti, created_at	JWT identifiers that have been revoked/blocked.

Relationships (conceptual)

• One organizer can own many events; each event references one venue.

• A student can have ticket records and waitlist entries for different events.

• A ticket references one event and one student; a check-in record references a ticket and the organizer who scanned it.

The application uses string identifiers for several document references. Confirm that all foreign-reference fields remain consistent if the schema is extended.

8. API overview

Prefix / endpoint examples	Purpose	Access
/api/health	Database connectivity check.	Public
/api/auth/register, /login, /refresh, /logout, /me	Registration, login, token renewal, logout and current-user lookup.	Public or token-protected as appropriate
/api/student/events, /events/, /events//book	Event discovery, detail and booking/waitlist entry.	Student JWT
/api/student/tickets, /tickets/, /events//waitlist	Ticket history, ticket cancellation and leaving a waitlist.	Student JWT
/api/organizer/events, /events/, /events//submit	Organizer event CRUD where permitted and submission for review.	Organizer JWT
/api/organizer/events//dashboard, /check-in	Event metrics and ticket check-in.	Organizer JWT
/api/organizer/venues	List venues for event creation.	Organizer JWT
/api/admin/events/pending, /events//approval	Review queue and event decision.	Admin JWT
/api/admin/venues, /venues/, /users, /users//role	Venue and user/role administration.	Admin JWT

This is a route-family summary, not a complete API contract. Request fields, response schemas, status codes and error formats should be documented and tested against the running application before external integration.

9. Non-functional requirements

Security

Passwords must remain hashed; tokens must be validated; protected actions must enforce role and ownership checks; secrets must be kept outside source control; API inputs should be validated and error responses should avoid exposing internal details.

Integrity and concurrency

Seat counts, ticket creation/cancellation and waitlist promotion must stay consistent under simultaneous requests. Transaction behavior depends on MongoDB being configured to support transactions (typically a replica set or sharded cluster).

Performance

Common event listings and lookup operations should respond within an agreed target under expected load. Add indexes based on measured query patterns; do not assume performance without load testing.

Usability and accessibility

Provide clear validation/error messages, distinguish loading/empty/error states, support keyboard navigation and readable contrast, and ensure the ticket/check-in flow is usable on typical laptop and mobile-sized screens.

Reliability

The health endpoint should indicate database availability. Operational deployment should include logs, backups, restart policy and monitoring; these should be verified separately.

Maintainability

Keep UI pages, API routes, auth/role checks and data operations organized by responsibility. Define a consistent API contract and add automated tests for business rules.

Portability

Docker-based setup should make local startup repeatable, with configuration provided through environment variables.

10. Assumptions, constraints and risks

Assumptions

• Users have a modern browser and network access to the application.

• MongoDB is available and configured with the capabilities required by transaction-based operations.

• Only administrators can grant elevated roles through the intended administrative workflow.

• Event and ticket dates are interpreted consistently by frontend and backend.

Risks and items to verify

• Secrets: the archive includes a backend .env path. Verify that it contains no real credentials before sharing or committing the project; rotate any exposed secret and keep .env out of version control.

• Time handling: source code mixes local datetime operations and UTC-oriented checks. Standardize storage and comparison on UTC-aware timestamps and test timezone boundaries.

• Waitlist positions: concurrent joins/cancellations may cause position races unless operations are made atomic or carefully transaction-protected.

• Capacity counter: verify that seats_sold is defined consistently as occupied seats across booked/used states and is reconciled after cancellation/check-in flows.

• Venue deletion: verify that deleting a venue referenced by an event is blocked or handled safely.

• Input validation: validate email format, password policy, tags shape, event fields, ticket codes and date filters consistently.

• Frontend/backend deployment: Docker Compose service networking, Vite host binding and production serving configuration should be tested in the actual target environment.

• Automated tests: no test suite was identified in the inspected source file list; add tests before relying on the system for real events.

> These are review findings and verification recommendations, not assertions that every listed issue is definitely exploitable or currently failing.

11. Acceptance criteria and test scenarios

ID	Test scenario
AT-01	Register with valid details; verify student role is assigned and password hash is stored, not plaintext.
AT-02	Attempt duplicate email registration; verify the request is rejected with a controlled error.
AT-03	Login with wrong password; verify access is denied and no token is issued.
AT-04	Call an admin endpoint as a student; verify access is denied.
AT-05	Create an event with end time before start time or capacity above venue capacity; verify validation failure.
AT-06	Submit an event; verify status becomes pending and it is not shown in the student feed before approval.
AT-07	Approve an event as admin; verify it becomes visible to students.
AT-08	Book available seats up to capacity; verify no overbooking and unique active tickets.
AT-09	Attempt a duplicate booking; verify the second request is rejected.
AT-10	Book a full event; verify waitlist entry and queue position are returned.
AT-11	Cancel a booked ticket with a waitlist; verify seat accounting and promotion of the first waiting student.
AT-12	Check in a valid ticket for the organizer's event; verify status becomes used and a second scan is rejected.
AT-13	Attempt check-in for another organizer's event; verify it is forbidden.
AT-14	Verify dashboard totals against known ticket and waitlist records.
AT-15	Stop or misconfigure MongoDB; verify the health endpoint returns an unavailable response rather than success.

12. Suggested future improvements

• Create a formal OpenAPI/Swagger specification for request/response schemas and status codes.

• Add automated unit and integration tests, especially for concurrency, capacity limits, role boundaries and waitlist promotion.

• Add audit history for event decisions, role changes and ticket operations.

• Standardize UTC date storage and formatting across frontend/backend.

• Add pagination and server-side limits for event, user and ticket listings.

• Add email notifications for approvals, booking confirmations, cancellations and waitlist promotion.

• Add event image upload/storage rules and validation instead of relying only on optional banner URLs.

• Add observability: structured logs, request IDs, error monitoring and database backup/restore drills.

• Improve deployment hardening: production WSGI server, HTTPS, secure CORS policy, rate limiting and secret management.

• Add reconciliation tools to detect and correct differences between event capacity counters, ticket records and waitlist records.

Conclusion

The project is a role-based campus event management platform with the core flow of event creation, approval, booking, waitlisting and check-in. Its key technical ideas are separation of frontend and backend, JWT authentication with role-based authorization, MongoDB persistence, and capacity-aware ticket workflows. The most important next steps are to verify concurrency and date handling, keep secrets safe, define a stable API contract, and test the critical business rules before production use.

Prepared based on source files in the supplied capstone archive. Requirements should be reviewed with the project owner and updated if the intended product scope differs from the implementation.